

PRACTICAL WEEK - 6

TASK 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    int fd_read, fd_write;

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started. Waiting for client...\n");

    fd_read = open(FIFO1, O_RDONLY);

    read(fd_read, message, sizeof(message));
    close(fd_read);

    printf("Server received: %s\n", message);

    fd_write = open(FIFO2, O_WRONLY);

    char response[] = "Message received by server.";
    write(fd_write, response, strlen(response) + 1);

    close(fd_write);

    printf("Server response sent.\n");

    unlink(FIFO1);
    unlink(FIFO2);

    return 0;
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano p-6.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc p-6.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./a.out
Server started. Waiting for client...
|
```

Code for client:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    char response[100];
    int fd_write, fd_read;

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    message[strcspn(message, "\n")] = '\0';

    fd_write = open(FIFO1, O_WRONLY);

    write(fd_write, message, strlen(message) + 1);
    close(fd_write);

    fd_read = open(FIFO2, O_RDONLY);

    read(fd_read, response, sizeof(response));
    close(fd_read);

    printf("Server response: %s\n", response);

    return 0;
}
```

```
Server started. Waiting for client...
Server received: hello server
Server response sent.
```

TASK 2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSRI. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIF01 "client_to_server"
#define FIF02 "server_to_client"

int main() {
    char message[100];
    char response[100];
    int fd_write, fd_read;

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    message[strcspn(message, "\n")] = '\0';

    fd_write = open(FIF01, O_WRONLY);

    write(fd_write, message, strlen(message) + 1);
    close(fd_write);

    fd_read = open(FIF02, O_RDONLY);

    read(fd_read, response, sizeof(response));
    close(fd_read);

    printf("Server response: %s\n", response);

    return 0;
}
```

```

keerthi_priya@LAPTOP-OSGVCE7A:~$ cd /mnt/c/JavaPrograms
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ls
Arithmetic.class      'Int variable.java'    SubMul.class
Arithmetic.java       'Int variables.java'   SubMul.java
Arithmetic.txt        Jhansi.class           SumDemo.class
Array_output.java    Jhansi.java            SumDemo.java
Calci.class          Keerthi.Html           SumDemo3.java
Calci.java           Leapyear.class         SumMatrix.class
Datatypes.class      Leapyear.java          SumMatrix.java
Datatypes.java       MatrixTranspose.class  Sumeven.class
Demo.class           MatrixTranspose.java   Sumeven.java
Demo.java            MethodOverLoading.java Sumodd.class
Divmod.class         MethodOverLoading.txt  Sumodd.java
Divmod.java          Min.java               TestDemo.class
Edowhile.class       New.class              TestDemo.java
Edowhile.java        New.java               Three.class
Efor.class           ObjectDemo.class       Three.java
Efor.java            ObjectDemo.java        child.class
Efor.txt             Operator.class         even.java
Equal.class          Operator.java          floating.java
Equal.java           Peven.class            p-6.c
Even.class           Peven.java             
Ewhile.class        Positive.class         parent.class
Ewhile.java         Positive.java          print.java
Ewhile.java         Postive.class          vinnu.class
FactorialDemo.class  Print.class            vinnu.java
FactorialDemo.java   ScanDemo.class         vote.class
Floating.class       ScanDemo.java          vote.java
Grades.class        StringDemo.class
Grades.java         StringDemo.java
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc p-6.c -o p62
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./p62
Signal handling program started.
PID: 721
Waiting for signals...
^C
SIGINT received (Ctrl+C).

SIGUSR1 received.

SIGTERM received.

```